

National Institute of Technology, Rourkela
End Semester Examination, Summer 2023
Compiler Design (CS3007)
5th semester B.Tech in Computer Science & Engineering

Duration: 3 Hours
Number of pages: 2

Full Marks: 50

- Attempt all questions. Q1 – Q8 are in page 1, Q9 and Q10 are in page 2.
 - If you answer a question, all sub questions should form part of it.
 - For each grammar $[a - z]^+$, $[0 - 9]^+$, ϵ , $+$ and $-$ are terminals while $[A - Z]^+$ are non-terminals. The starting symbol is the non-terminal associated with the first grammar rule.
- Q 1. Write the compilation steps involved in translating the high level language statement “circumference = 2*pi*radius;” to its equivalent target code where **circumference** is a **float** variable, symbolic constant **pi**=3.14 and **radius** is an **int** variable. [5]
- Q 2. Construct a deterministic finite automata (DFA) to accept the following regular expressions and return tokens
if {return IF}
int {return INT}
[fint]+ {return IDENTIFIER}
Provide a 2-D array representation of the DFA. [3+2]
- Q 3. Compress the 2-D array representation for the DFA asked in Q2 using four arrays
base array - determines base location of entries for state s ($base[s]$),
next array - entry for next state of s on input symbol offset a ($next[base[s] + a]$),
check array - stores $check[base[s] + a] = s$ for a valid transition $\delta(s, a)$,
default array - alternative base location if $check[base[s] + a] \neq s$.
Consider $s \in \{0, 1, 2, \dots\}$ and offset values (a) for symbols i, f, n and t are 0, 1, 2 and 3, respectively.
How much reduction in space is achieved ? What is the token acceptance time ? [3+1+1]
- Q 4. For the following grammar with S as starting symbol, $S \rightarrow Aa \mid b, A \rightarrow Ac \mid Sd \mid e$
(a) Justify the possibility of constructing a recursive descent parser. [2]
(b) Transform the grammar to another equivalent form so that the construction of recursive descent parser is possible. [3]
- Q 5. For the following grammar with A as starting symbol, $A \rightarrow Bb \mid Cd, B \rightarrow aB \mid \epsilon, C \rightarrow cC \mid \epsilon$
(a) Find the FIRST and FOLLOW sets of each of the nonterminals. [4]
(b) Verify that the grammar is LL(1). [1]
- Q 6. Considering the grammar in Q5
(a) Construct the predictive parser table for it. [3]
(b) Give a trace of parsing for the input string “ccd” using appropriate actions. [2]
- Q 7. Show that the grammar $S \rightarrow aAd \mid bBd \mid aBe \mid bAe, A \rightarrow c, B \rightarrow c$ is not LALR(1). [5]
- Q 8. Consider the grammar $NUMBER \rightarrow SIGN LIST, SIGN \rightarrow + \mid -, LIST \rightarrow LIST BIT \mid BIT, BIT \rightarrow 0 \mid 1$. The attributes with the grammar symbols are as follows
NUMBER - $NUMBER.value \in \{0, 1, 2, \dots\}$
SIGN - $SIGN.negative \in \{false, true\}$
LIST - $LIST.position \in \{0, 1, \dots\}, LIST.value \in \{0, 1, 2, \dots\}$
BIT - $BIT.position \in \{0, 1, \dots\}, BIT.value \in \{0, 2^{BIT.position}\}$.
(a) Design a syntax directed translation scheme for converting a sign-magnitude binary number to its equivalent decimal form. [3]
(b) Construct the evaluation tree for the input string “-1100”. [2]

Q 9. Consider the program given below, in a block-structured pseudo-language with lexical scoping and nesting of procedures permitted.

```

01. Program main;
02.   Var ...
03.   Procedure A1;
04.     Var ...
05.     Call A2;
06.   End A1
07.   Procedure A2;
08.     Var ...
09.     Procedure A21;
10.       Var ...
11.       Call A1;
12.     End A21
13.     Call A21;
14.   End A2
15.   Call A1;
16. End main.

```

Give the correct set of activation records along with their access links. [5]

Q 10. Given a 32-bit processor with general purpose register set $R = \{r_0, r_1, r_2, r_4, r_5, r_6, r_7\}$. The instruction set of the machine is as follows

LOAD X,Y; $X \leftarrow [Y]$ – loads value in a location Y into register X.

STORE X,Y; $[X] \leftarrow Y$ – stores the value in register Y into the location X.

MOV X,Y; $X \leftarrow Y$ – moves the value in register Y into the register X.

ADD X,Y,Z; $X \leftarrow Y + Z$ SUB X,Y,Z; $X \leftarrow Y - Z$ MUL X,Y,Z; $X \leftarrow Y * Z$ DIV X,Y,Z; $X \leftarrow Y / Z$

where X and Y are registers, Z is either a register or an immediate value

CMP X,Y - compares the value in register X with a register or an immediate value Y.

JLT L; goto to level L if $X < Y$ after the execution of CMP X,Y.

LSHF X; left shifts register X by one bit RSHF X; right shifts register X by one bit

Using the instruction set write equivalent 3-address code for the following statements. [5]

(a) $x = y - z$;

(b) $a = a / 2$;

(c) $b = a[i]$;

(d) $*p = y$;

(e) if $x < y$ goto L;

*****All the best*****